

DATABASE MANAGEMENT SYSTEMS

Design and Implementation of a Database Management System for Bus Reservation and Fleet Scheduling System

PROJECT REVIEW 2

Presented by: Mangala Samyuktha (Roll No. 25WU0102258)

Project Recap & Review 2 Objectives

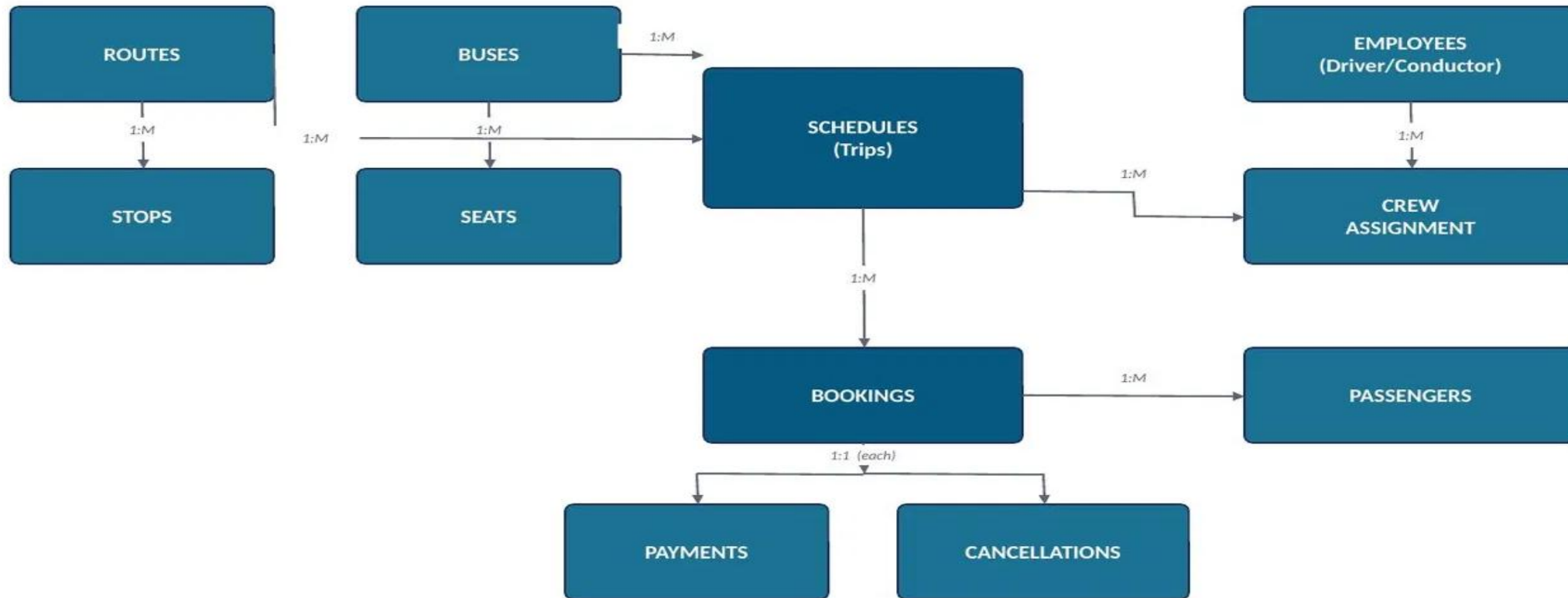
The problem

Bus operators manage routes, stops, vehicles, seats, schedules, crews, bookings, cancellations and fares. Manual booking can allocate the same seat twice, and crew duties can silently overlap. We are building a relational database + a small Java/Python front end to remove both problems.

What Review 2 demonstrates

- Normalized relational schema (up to 3NF) with a data dictionary
- DDL with primary keys, foreign keys, UNIQUE, NOT NULL, CHECK, DEFAULT
- Realistic sample data across all 11 tables
- SQL: joins, nested sub-queries, aggregation, and reporting views

Conceptual ER Diagram



Legend: dark boxes = transaction tables | arrows point from "1" side to "many" side

Key business rules captured in the model: one booking per seat per trip • no overlapping crew duties • valid departure/arrival times • one payment / one cancellation per booking

Normalization — Up to 3NF

Why normalize?

A single flat 'Booking' sheet repeats route, bus and passenger details on every row — causing update/insert/delete anomalies. We decompose it using functional dependencies.

- 1NF

Atomic attributes only
e.g. no comma-separated stop lists inside Routes — Stops pulled into its own table
- 2NF

No partial dependency on a composite key
Seats depends fully on (bus_id, seat_number), not on bus_id alone
- 3NF

No transitive dependency
Route source/destination stored once in Routes, not repeated inside Schedules or Bookings

Final table set (11 relations)

Routes (PK route_id)
Stops (PK stop_id, FK route_id)
Buses (PK bus_id)
Seats (PK seat_id, FK bus_id)
Employees (PK emp_id)
Schedules (PK trip_id, FK route_id, bus_id)
Crew_Assignment (PK assignment_id, FK trip_id, emp_id)
Passengers (PK passenger_id)
Bookings (PK booking_id, FK trip_id, seat_id, passenger_id)
Cancellations (PK cancellation_id, FK booking_id)
Payments (PK payment_id, FK booking_id)

DDL & Integrity Constraints

Constraint types used

PRIMARY KEY	Every table (AUTO_INCREMENT surrogate keys)
FOREIGN KEY	Routes→Stops, Buses→Seats, Schedules→Bookings, etc.
UNIQUE	bus_number, (bus_id,seat_number), (trip_id,duty_role)
NOT NULL	All mandatory business attributes
CHECK	distance_km > 0, arrival_time > departure_time, valid status enums
DEFAULT	status = 'Scheduled' / 'Confirmed', booking_date = CURRENT_TIMESTAMP
TRIGGERS	No double seat booking • No overlapping crew duty (beyond plain SQL constraints)

Sample DDL (Schedules table)

```
CREATE TABLE Schedules (  
  trip_id      INT AUTO_INCREMENT PRIMARY KEY,  
  route_id     INT NOT NULL,  
  bus_id       INT NOT NULL,  
  trip_date    DATE NOT NULL,  
  departure_time DATETIME NOT NULL,  
  arrival_time  DATETIME NOT NULL,  
  base_fare    DECIMAL(8,2) NOT NULL  
              CHECK (base_fare > 0),  
  status       VARCHAR(15) NOT NULL  
              DEFAULT 'Scheduled'  
              CHECK (status IN ('Scheduled',  
                                'Departed', 'Completed', 'Cancelled')),  
  FOREIGN KEY (route_id) REFERENCES Routes(route_id),  
  FOREIGN KEY (bus_id)   REFERENCES Buses(bus_id),  
  CHECK (arrival_time > departure_time)  
);
```

Sample Data

Populated across all 11 tables (3 routes, 3 buses, 4 crew, 3 passengers, 3 trips, 3 bookings, cancellations & payments). Sample below:

booking_id	passenger	route	trip_date	seat	fare	status
1	Mangala Samyuktha	Hyderabad → Vijayawada	2026-09-20	A1	450	Confirmed
2	Arjun Reddy	Hyderabad → Vijayawada	2026-09-20	A2	450	Cancelled
3	Priya Sharma	Hyderabad → Bengaluru	2026-09-20	B1	900	Confirmed

Business rule enforced live: re-inserting seat A1 for trip 1 raises “Seat already booked for this trip” via trigger — will demonstrate on request.

Employees & Crew Assignment (excerpt)

Ravi Kumar (Driver) + Anitha Rao (Conductor) → Trip 1 (Hyd→Vijayawada, 06:00–10:30)
Suresh Reddy (Driver) + Kiran Babu (Conductor) → Trip 2 (Hyd→Bengaluru, 20:00–06:00)
Ravi Kumar (Driver) + Anitha Rao (Conductor) → Trip 3 (Vijayawada→Chennai, next day 07:00) — no time overlap, so allowed

SQL Queries — Joins, Sub-queries, Aggregation

JOIN	Full booking detail	<code>Bookings ⌘ Passengers ⌘ Schedules ⌘ Routes ⌘ Seats</code>
AGGREGATION	Revenue per route	<code>SUM(fare_amount) GROUP BY route (via vw_route_revenue)</code>
AGGREGATION	Trip occupancy %	<code>COUNT(bookings)/total_seats GROUP BY trip (via vw_trip_occupancy)</code>
NESTED SUB-QUERY	Passenger who paid the highest fare	<code>passenger_id IN (SELECT ... WHERE fare_amount = (SELECT MAX(fare_amount)...))</code>
NESTED SUB-QUERY	Trips with zero bookings	<code>trip_id NOT IN (SELECT DISTINCT trip_id FROM Bookings)</code>
JOIN (3-way)	Crew duty roster	<code>Crew_Assignment ⌘ Schedules ⌘ Routes ⌘ Employees</code>

Views — Sample Output

vw_route_revenue

route	total_revenue
Hyderabad → Vijayawada	450
Hyderabad → Bengaluru	900

vw_trip_occupancy

trip	seats_booked	occupancy %
Hyd→Vijayawada	2 / 40	5.0%
Hyd→Bengaluru	1 / 50	2.0%

Why views?

Views encapsulate the join + aggregation logic once, so the future Java/Python front end can simply query “SELECT * FROM vw_trip_occupancy” for its dashboard instead of repeating complex SQL in application code.



Thank You